

Giving your projects a memory.

With busy days for each of us, you may find that parts of your AI-involved work start to drift, or produce gaffes, as chats, iterations and your ideas move along. The bots hallucinate: 'tis how it is. Follow this guide to learn how to manage your projects, and give them a bit of a memory.

You'll need

- ✓ **Claude Code**, signed in claude.com ↗
- ✓ **Obsidian**, free, to read your vault obsidian.md ↗
- ✓ a **Mac** or a **Windows** PC
- ✓ about **10 minutes**, once

A vault is your project's memory.

When a project spans days, your decisions, your thinking, your whole sense of what you're even doing can erode, drift off, or cease to exist altogether. If that weren't bad enough, imagine being a sycophantic robot going through the same thing. *L'horreur.*

To keep things from dissipating out of memory, or scrolling out of view forever, the **vault** offers some relief: it keeps an AI workspace organised.

A vault, read in **Obsidian** (though any tidy folder of text files works), does a few things for the AI you work with:

- gives the AI a dedicated place to keep memories, notes, and iterations
- holds it to a bit of digital hygiene
 - fewer hallucinations, more accurate statements and assertions
 - far easier to read when you review it by hand

- interlinks data, notes, and information (à la wikilinks)
- keeps a persistent memory of how the project grew
- lets you pick up cold: a new chat can be caught up in one message
 - no re-explaining the project from scratch every Monday
- settles arguments with your past self: the decision and its reasoning sit in writing
- keeps the receipts when someone asks why a thing was built that way
- costs you less: a short, pointed catch-up beats re-pasting half a project
 - fewer tokens burned re-establishing what the AI should already know
- it is just folders and text files: yours, readable without us, portable anywhere



What lives in a vault?

Files and folders. Folders and files. That's it. In terms of what they're *for*:

Sessions Automatic notes of what you did, per day	Decisions The choices made, and why
Handoff Where things stand, for next time	Board A JIRA-like board, but less annoying. Kanban-style

How it fits together



And it reads back when you ask `sitrep` or `check`.

What a connected project looks like

```
Claude Vault/  
└─ projects/  
  └─ my-app/  
    ├── brief.md           the project's purpose  
    ├── sessions/         one note per working day  
    ├── decisions/        why you chose things  
    ├── tasks/             → becomes the board  
    ├── board/            the kanban, saved to disk  
    └─ _handoff.md        where things stand
```

Adjutant keeps it fresh. You just work. Mainly.

The adjutant is a spin-off from a personal framework I've built up working with the bane of my existence: AI. So far I've found it mostly reliable. As with any AI plug-in, how well it serves you tracks your own willingness to catch it misbehaving as you go.

That said, it makes two things easier: directing what the AI does, and bringing a touch of systems thinking to how you work.

Happens on its own	What you must do
■ Session notes ■ The handoff, kept current ■ The board, built from your in-chat task lists ■ Every write checked against a standard shape	■ Connect it once per project ■ Run sitrep when you're back, or starting a new chat ■ Remind the chat to use the adjutant to capture insight and progress ■ Run tidy to clear the machine dust

The adjutant has seven **verbs**

Verbs are commands you drop into a chat message. Type one and it triggers that adjutant feature.

connect	Link a project to its vault
sync	Push current state to the vault
check	Health report READ ONLY
sitrep	“Where was I?” after a break READ ONLY
tidy	Safe cleanup, previews first
dream	Advisory review READ ONLY

On dream, and on cost

dream reads your notes and hands you a report — what's stale, what contradicts, what's orphaned — then **stops**. It changes nothing on its own; you decide. And the whole plugin is built to be light: heavy reads estimate their cost and ask first.



Screenshot placeholder · the adjutant at work in the Claude Code CLI

Set it up.

First, install the two free apps this needs: **Claude Code** and [Obsidian](#). Downloading Obsidian from [obsidian.md](#) works on both a Mac and a PC.

On a Windows PC

The steps below are the same on both systems, with one extra setup on Windows: Claude Code runs inside **WSL**, a Linux-flavoured terminal that ships with Windows. Open **PowerShell**, run **ws1 -install** once, restart, then open **Ubuntu** from the Start menu and do everything there. Your vault can live in **OneDrive** as usual: it gets found either way.

Each box below has a **Copy** button. Copy it, then paste it where the label says — your Terminal app, or the Claude Code chat. You don't type the box itself.

1 Get access

The repo is private, so GitHub has to let Claude Code in. You do this once per computer.

In your Terminal app signs you in to GitHub

```
gh auth login
```

2 Install the plugin

Paste these into the Claude Code chat, one at a time.

In Claude Code tells it where to find the tool

```
/plugin marketplace add TomVDH/furtive-follies
```

In Claude Code installs adjudant

```
/plugin install adjudant
```

3 Connect your project

Do this once inside each project. It shows where a vault can live, recommends a spot, and makes it for you.

In Claude Code sets up the vault

```
/adjudant connect
```

4 Open it in Obsidian

connect makes the vault folder for you. In Obsidian, choose **Open folder as vault** and pick that folder — now you can read your notes, follow the links between them, and see the board.

Where your vault should live

Cloud-sync folder – recommended

iCloud, OneDrive, Google Drive, Dropbox. Your notes follow you across computers.

Local folder

~/Documents. Fine if you only ever use one computer.

✓ **That's it.** The record-keeping runs from here. Come back and ask `/adjudant sitrep` after a break.

The terminal, in ten commands.

The terminal is a text way to tell your computer what to do. You point it at a folder, then run a command. These ten cover almost everything a beginner needs. Type

one, press Enter.

<code>pwd</code>	Show the folder you're in right now	<code>pwd</code>
<code>ls</code>	List what's in this folder	<code>ls</code>
<code>cd</code>	Go into a folder — or <code>cd ..</code> to go up one	<code>cd projects</code>
<code>mkdir</code>	Make a new folder	<code>mkdir notes</code>
<code>cp</code>	Copy a file	<code>cp a.md b.md</code>
<code>mv</code>	Move a file, or rename it	<code>mv old.md new.md</code>
<code>rm</code>	Delete a file. No undo — see <code>trash</code> below	<code>rm draft.md</code>
<code>cat</code>	Print a file to the screen	<code>cat notes.md</code>
<code>less</code>	Read a long file a page at a time — <code>q</code> quits	<code>less big.log</code>
<code>clear</code>	Wipe the screen clean	<code>clear</code>

Not sure what a command does? Run `tlldr <command>` for a plain, example-first reminder (it comes with the suitcase, below).

The suitcase: a friendlier terminal.

The bundle ships a small kit of command-line tools — the **suitcase**. It makes the terminal nicer without changing how anything works, and it's optional. The check run below changes nothing; the second one installs. Anything you already have is skipped.

In your Terminal app · inside the downloaded folder

a dry run — shows what it would do

```
bash onboarding/onboard.sh --check
```

In your Terminal app

installs the tools for real

```
bash onboarding/onboard.sh
```

What each tool does

ALTERS means the same command now gives nicer output. **ADDS** means a new command that sits beside your old ones. **ls**, **cd**, **rm**, **grep** and **find** stay exactly as they were — nothing you already know stops working.

z ADDS	Jump to a folder by name, from anywhere	<code>cd ../../work/app → z app</code>
ll ADDS	A rich listing: sizes, dates, git state (1 and 1t too)	<code>ls -lah → ll</code>
cat ALTERS	Same command, now with color and line numbers. Stays plain when piped	<code>cat app.py → cat app.py</code>
rg ADDS	Search inside files, fast	<code>grep -rn TODO . → rg TODO</code>
fd ADDS	Find files by name	<code>find . -name "*.md" → fd .md</code>
git diff ALTERS	Same command, now a clear colored side-by-side	a wall of +/- lines → a readable diff
trash ADDS	Delete to the Trash, not forever. <code>rm</code> is left alone	<code>rm note.md (gone) → trash note.md</code>
glow ADDS	Read a markdown file, nicely rendered	<code>cat notes.md → glow notes.md</code>
tldr ADDS	Plain, example-first help for any command	<code>man tar → tldr tar</code>

sl and **cbonsai** earn their place by adding nothing useful — a steam train and a bonsai tree, for when the terminal needs a smile.

Recommended terminal setup

REQUIRED	Zsh · zsh.org The shell these shortcuts assume. Already the default on a Mac, and on Windows inside Ubuntu. Nothing to do unless you changed it yourself.	<code>already set up</code>
RECOMMENDED	iTerm2 · item2.com A nicer terminal than the built-in one. Repo: gnachman/iTerm2	<code>brew install --cask iterm2</code>
RECOMMENDED	Powerlevel10k · romkatv/powerlevel10k A fast, informative prompt. Install it, add it to <code>~/.zshrc</code> , then run <code>p10k configure</code> . Install guide →	<code>brew install powerlevel10k</code>

Optional: a live statusline

The bundle also ships a small statusline. It shows your git branch, the vault this project is linked to, and how full the context window is — a quiet reminder to keep sessions short. Turn it on with one line in `~/.claude/settings.json`, using the real path to the script.

In `~/.claude/settings.json` turns the statusline on

```
"statusLine": { "type": "command", "command": "bash ~/furtive-follies/onboard" }
```

Point the path at wherever you put the folder. It uses **jq** or **python3** to read the status; with neither, it still shows the git part.

On a **Windows** PC, run that same installer inside **Ubuntu** (the WSL terminal from the setup section). It sorts out the rest itself, and skips anything you already have.

If something looks off.

Vault not found Ask <code>/adjudant connect</code> again — it re-links the project.	A write got blocked The message names the field to fix. Adjust it, write again.	No board yet It's born on your first task note. No tasks, no board.
--	---	--